

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчёт о программном проекте на тему:

**«Построение эмбедингов пользователей по последовательности игровых событий на основе
контрастивного обучения, трансформеров и RNN»**

(часть проекта: метод RNN)

Выполнили:

студент группы БПМИ 245

Никонов Фёдор Алексеевич

31.05.2026

студент группы БПМИ 249

Гринюк Илья Олегович

31.05.2026

студент группы БПМИ 2414

Петраков Алексей Сергеевич

31.05.2026

Принял руководитель проекта:

Мамаев Александр Александрович

руководитель службы ML

ООО «Яндекс.Такси»

Москва, 2026

Содержание

Аннотация	4
Ключевые слова	4
1 Введение	5
2 Обзор литературы	5
3 Постановка задачи	6
3.1 Неформальная постановка	6
3.2 Формальная постановка	6
4 Данные и предобработка	7
4.1 Источник данных	7
4.2 Очистка и построение последовательностей	8
4.3 Разбиение выборки	8
4.4 Два режима построения эмбедингов	9
5 Метрики и оценка в downstream-задачах	9
6 Модель построения эмбедингов	10
6.1 Слой эмбедингов событий	10
6.2 Рекуррентный энкодер	10
6.3 Обучение через next-event prediction	10
6.4 Pooling скрытых состояний	11
7 Экспериментальное исследование	11
7.1 Next-event prediction	11
7.2 Оценка удержания prefix-based	11
7.3 Pooling и capacity	13
7.4 Supervised fine-tuning и negative controls	14
7.5 Full-history сравнение	15
7.6 Командное сравнение подходов full-history	15
7.7 Анализ пространства эмбедингов	16
8 Воспроизводимость и проверка корректности	17
9 Заключение	18

Аннотация

Работа посвящена построению эмбедингов пользователей мобильной игры по последовательностям игровых событий. Эмбединг должен сжимать историю действий пользователя в вектор фиксированной размерности, который можно использовать в downstream-задачах: прогнозировании удержания, анализе поведения и сравнении пользовательских профилей.

В этой части группового проекта исследуется подход на основе рекуррентных нейронных сетей. Были реализованы GRU- и LSTM-энкодеры, обучаемые без разметки удержания на задаче next-event prediction. После обучения скрытые состояния RNN используются для построения пользовательских эмбедингов. Полезность полученных представлений проверяется на задаче предсказания удержания.

В работе рассмотрены два режима оценки. В prefix-based режиме модель получает только начальную часть истории пользователя, а метка удержания считается относительно конца префикса. Этот режим нужен для строгой проверки без утечки будущего поведения в признаки. В full-history режиме эмбединг строится по последним 512 событиям полной истории пользователя; он добавлен для сопоставления с общим форматом командных экспериментов.

RNN-модели значительно превосходят простые baseline в предсказании следующего события: на тестовой выборке GRU достигает MRR 0.8885 и Hit@10 0.9926. В строгой prefix-based проверке финальный кандидат GRU h256, 1 layer, max pooling даёт ROC-AUC 0.6922 ± 0.0052 для retention 7d и 0.7051 ± 0.0032 для retention 14d, улучшая табличные baseline-признаки. В согласованном full-history режиме RNN-эмбединги отдельно достигают ROC-AUC 0.9413 для retention 7d.

Ключевые слова

Эмбединги пользователей, последовательности событий, рекуррентные нейронные сети, GRU, LSTM, обучение без ручной разметки, next-event prediction, предсказание удержания, оценка prefix-based, оценка в downstream-задачах.

1 Введение

Поведение игрока в мобильной игре можно описать последовательностью событий: запуск игры, переходы между экранами, получение наград, взаимодействие с рекламой, игровые действия и другие элементы пользовательской активности. Такая последовательность содержит информацию о стиле игры, вовлеченности и вероятности возвращения пользователя.

Прямое использование сырых последовательностей в прикладных моделях неудобно. У разных пользователей сильно различается длина истории: в подготовленных данных медианная длина равна 83 событиям, а максимальная длина достигает 185 204 событий. Кроме того, порядок событий несёт смысл, который теряется при простом подсчёте частот. Поэтому полезно построить отображение, переводящее историю пользователя в компактный вектор фиксированной размерности.

Цель моей части проекта - построить такие пользовательские эмбединги с помощью RNN-моделей и проверить, сохраняют ли они полезный поведенческий сигнал. В отличие от модели, обучаемой непосредственно на удержание, RNN-энкодер обучается на последовательной структуре самих событий: по префиксу истории он предсказывает следующее событие. Удержание используется позже только как внешний критерий качества эмбедингов.

Работа является частью группового проекта, в котором сравниваются три подхода к построению эмбедингов пользователей: SimCLR, BERT4Rec/Transformer и RNN. Настоящий отчет описывает RNN-часть: подготовку последовательностей, обучение GRU/LSTM, экспорт эмбедингов, оценку в downstream-задачах и дополнительные проверки устойчивости.

Основные результаты:

- реализован полный пайплайн для RNN-эмбедингов: предобработка, user-level split, обучение на предсказании следующего события, экспорт эмбедингов, оценка удержания;
- GRU и LSTM сильно превосходят MostPopular и Markov-1 в задаче next-event prediction;
- prefix-based протокол показывает, что RNN-эмбединги дают умеренный, но устойчивый прирост к табличным признакам в задаче удержания;
- проведены проверки способов pooling скрытых состояний, ёмкости модели, дообучения с разметкой, отрицательных контролей и устойчивости к случайной инициализации;
- дополнительно построена RNN-ветка full-history на общем командном split для будущего сопоставления с SimCLR и BERT4Rec.

2 Обзор литературы

Рекуррентные нейронные сети являются классическим инструментом для моделирования последовательностей. LSTM была предложена Hochreiter и Schmidhuber [6] для борьбы с проблемой исчезающих градиентов в длинных последовательностях. Позже Cho и соавторы предложили GRU

[4], более компактную рекуррентную архитектуру с gating-механизмом. Обе архитектуры применимы к пользовательским событиям: модель последовательно обновляет скрытое состояние и тем самым сжимает историю пользователя в вектор.

В рекомендательных системах последовательное моделирование часто применяется для предсказания следующего действия или следующего объекта. Работа Hidasi и соавторов [5] показала, что GRU-подход может эффективно моделировать session-based поведение. Для более поздних transformer-based методов важной точкой сравнения является BERT4Rec [8], где последовательность элементов обучается через masked language modeling. В нашем групповом проекте BERT4Rec используется в другой ветке, а RNN-ветка служит более простой и интерпретируемой sequence-моделью.

Общая тема проекта связана с представлениями, обучаемыми без ручной разметки. В этой области важны contrastive predictive coding и InfoNCE-подход [7], а также SimCLR [2], применяющий контрастивное обучение для построения представлений без ручной разметки. Эти работы относятся прежде всего к другим веткам проекта, но задают общий контекст: эмбединги обучаются на внутренней структуре данных, а затем проверяются на внешних задачах.

Качество эмбедингов удобно проверять через downstream-задачи, то есть внешние задачи, не использовавшиеся напрямую при обучении представлений. В этой работе основной критерий такой проверки - предсказание удержания. Для нелинейной оценки признаков используется XGBoost [3], так как градиентный бустинг хорошо работает с табличными признаками и позволяет сравнивать разные семейства эмбедингов в единой процедуре оценки. Общая идея обучения представлений как построения признаков, полезных для downstream-задач, соответствует обзору Bengio и соавторов [1].

3 Постановка задачи

3.1 Неформальная постановка

Для каждого пользователя известна последовательность игровых событий, отсортированная по времени. Нужно построить модель, которая переводит эту последовательность в вектор фиксированной размерности. Пользователи с похожим поведением должны получать похожие векторы, а полученные эмбединги должны быть полезны в downstream-задачах.

Важная особенность работы: разметка удержания не используется как основной сигнал для обучения энкодера. Энкодер обучается на задаче next-event prediction, а удержание нужно для внешней проверки качества представлений.

3.2 Формальная постановка

Пусть пользователь u задан последовательностью событий

$$s_u = (x_1, x_2, \dots, x_{L_u}), \quad x_i \in V,$$

где V - словарь событий, а L_u - длина истории пользователя. Требуется обучить энкодер

$$f_\theta : V^* \rightarrow \mathbb{R}^d,$$

который переводит последовательность произвольной длины в эмбединг $z_u = f_\theta(s_u)$.

В RNN-подходе модель обучается на предсказании следующего события. Для обучающего окна $(x_1, \dots, x_t)$ модель предсказывает следующий токен x_{t+1} :

$$\hat{p}_{t+1} = \text{softmax}(Wh_t + b),$$

где h_t - скрытое состояние RNN после обработки первых t событий. После обучения скрытые состояния модели используются как представление пользователя.

Для downstream-проверки строится бинарная метка retention_h : был ли пользователь активен после горизонта h дней. В prefix-based режиме эта метка считается относительно конца префикса, а не относительно конца всей истории.

4 Данные и предобработка

4.1 Источник данных

Данные представляют собой логи AppMetrica мобильной игры. Каждая запись содержит идентификатор устройства пользователя, имя события, JSON-параметры события, timestamp и session id. Предобработка выполнялась на общем наборе логов проекта.

После очистки и фильтрации в RNN-ветке подготовлено 52 495 пользователей. Размер словаря событий после фильтрации составляет 81 токен. Распределение длин последовательностей имеет тяжелый хвост:

Таблица 1 – Характеристики подготовленных последовательностей RNN-ветки

Метрика	Значение
пользователей	52,495
размер словаря	81
минимальная длина	10
медианная длина	83

Метрика	Значение
средняя длина	972.12
95-й перцентиль	4,694
максимальная длина	185,204

4.2 Очистка и построение последовательностей

На этапе очистки удаляются записи с пустыми ключевыми полями, технические события и служебные debug/test/internal события. Далее события каждого пользователя сортируются по времени и row_id, чтобы получить стабильный порядок при совпадающих timestamp. Пользователи менее чем с 10 событиями исключаются.

Каждому имени события сопоставляется integer token, то есть целочисленный идентификатор события. Последовательность пользователя хранится как список таких токенов. Padding token с индексом 0 используется только для выравнивания длин последовательностей при подготовке batch для RNN.

4.3 Разбиение выборки

Финальные prefix-based эксперименты используют общий командный user-level split, импортированный из master_split_lesha.csv. Из 68 394 пользователей master split в RNN-подготовку попали 52 495 пользователей; все они присутствуют в master split.

Таблица 2 – Статистика последовательностей по частям user-level split

Split	Пользователей	Средняя длина	Медианная длина	Максимальная
				длина
train	36,760	956.73	84	131,726
val	7,896	984.92	79	63,480
test	7,839	1,031.39	86	185,204

Split проводится по пользователям, а не по событиям. Это исключает ситуацию, когда события одного пользователя одновременно попадают в train и test.

Для командного full-history сравнения отдельно использован согласованный протокол на SimCLR processed sequences. В нем RNN получает последние 512 событий для всех пользователей общего split: 47 875 train, 10 259 val и 10 260 test. Этот прогон не заменяет эксперименты prefix-based, а нужен только для сопоставления с SimCLR/BERT4Rec в ретроспективной постановке full-history.

4.4 Два режима построения эмбедингов

В работе используются два режима.

Первый режим - prefix-based. Модель строит эмбединг только по первым `prefix_len` событиям пользователя. Метка удержания считается относительно конца этого префикса. Такой режим отвечает на вопрос: можно ли по ранней истории пользователя построить представление, полезное для прогноза будущего поведения. Это строгая постановка без утечки будущих событий в признаки.

Второй режим - full-history. Модель строит эмбединг по последним 512 событиям полной доступной истории пользователя. Такой режим отвечает на другой вопрос: насколько хорошо RNN кодирует уже наблюдавшийся поведенческий профиль. Он добавлен для командного сопоставления с ветками, которые также строят full-history эмбединги.

Эти два режима не следует смешивать как один и тот же тип прогноза. Результаты prefix-based являются основной проверкой качества раннего прогноза без утечки, а результаты full-history - отдельным ретроспективным сравнением.

5 Метрики и оценка в downstream-задачах

Для задачи next-event prediction используются MRR и Hit@K. MRR отражает средний обратный ранг правильного следующего события, а Hit@K - долю примеров, в которых правильное событие попало в первые K предсказаний.

Для оценки удержания используются ROC-AUC и PR-AUC. ROC-AUC оценивает качество ранжирования положительных и отрицательных примеров, а PR-AUC дополнительно чувствителен к дисбалансу классов. Это важно для больших горизонтов удержания, где доля положительного класса мала.

Downstream-оценка проводится через бинарный классификатор XGBoost. Модель обучается на train-части, а метрики считаются на val/test. В таблицах устойчивости обучение повторяется при нескольких случайных инициализациях, после чего приводятся среднее значение и стандартное отклонение.

Baseline-признаки представляют собой простые агрегаты поведения пользователя. В prefix-based режиме они строятся только по тому же префиксу, что и RNN-эмбединг: длина префикса, число уникальных событий, доля повторов, число сессий, число активных дней и доли top-K частых событий. Top-K события выбираются только по train-части. В full-history режиме аналогичные признаки строятся по последнему окну истории пользователя.

В таблицах ниже «Baseline» обозначает только эти агрегированные признаки, «RNN embedding» - только вектор RNN-энкодера, а «Baseline + RNN embedding» - их конкатенацию. В

командном сравнении SimCLR/BERT/RNN используются результаты без дополнительных baseline-признаков, так как они лучше отражают качество самих методов построения эмбеддингов.

6 Модель построения эмбеддингов

6.1 Слой эмбеддингов событий

На вход RNN подается последовательность целочисленных идентификаторов событий. Слой nn.Embedding переводит каждый token в плотный вектор размерности event_dim. В основных экспериментах используется event_dim = 64, padding token имеет индекс 0 и не несет информации о событии.

6.2 Рекуррентный энкодер

В работе реализован общий RecurrentEncoder, который поддерживает GRU и LSTM. Энкодер получает последовательность эмбеддингов событий и обновляет скрытое состояние по времени. Для обучения используется pack_padded_sequence, поэтому padding не влияет на скрытые состояния реальных событий.

Базовые модели:

Таблица 3 – Основные конфигурации RNN-энкодеров

Модель	Размерность события	Размерность скрытого состояния	Слоёв	Dropout
Базовая GRU	64	128	1	0.0
Базовая LSTM	64	128	1	0.0
Финальный GRU-кандидат	64	256	1	0.0

6.3 Обучение через next-event prediction

Модель обучается предсказывать следующее событие. Для каждого обучающего окна входом является префикс, а целевой меткой - следующий token. На выходе RNN-проекция дает распределение по словарю событий.

Такая objective не использует разметку удержания и поэтому подходит для построения универсальных поведенческих эмбеддингов. Если модель хорошо предсказывает следующее событие, значит скрытое состояние содержит информацию о локальной структуре пользовательской истории.

6.4 Pooling скрытых состояний

Для получения одного вектора на пользователя из последовательности скрытых состояний используются несколько стратегий pooling:

- last: последнее скрытое состояние;
- mean: среднее по всем реальным событиям;
- max: покомпонентный максимум по реальным токенам, без учета padding;
- last_mean: конкатенация last и mean.

Ablation способа pooling показала, что для оценки удержания наиболее полезной оказалась max pooling. Финальный кандидат использует GRU h256, 1 layer с такой агрегацией.

7 Экспериментальное исследование

7.1 Next-event prediction

Сначала проверялось, умеют ли RNN-модели моделировать последовательность событий. Для сравнения использовались две простые baseline. MostPopular всегда ранжирует события по их частоте в train-части и не учитывает историю конкретного пользователя. Markov-1 использует только последнее событие и ранжирует следующие события по эмпирическим вероятностям переходов первого порядка.

Таблица 4 – Качество моделей на задаче next-event prediction

Модель	Split	MRR	Hit@5	Hit@10
MostPopular	test	0.4510	0.7154	0.8781
Markov-1	test	0.8158	0.9408	0.9775
GRU	test	0.8885	0.9733	0.9926
LSTM	test	0.8886	0.9732	0.9927

GRU и LSTM дают почти одинаковое качество и заметно превосходят простые baseline. Это подтверждает, что рекуррентная модель извлекает из истории пользователя больше информации, чем частотная модель или переход первого порядка.

7.2 Оценка удержания prefix-based

Основная проверка удержания проводилась в prefix-based режиме. Для каждого пользователя берутся первые prefix_len событий. Признаки и RNN-эмбеддинги строятся только по этому префиксу.

Пользователь включается в оценку по конкретному горизонту только если у него есть полный префикс и горизонт полностью наблюдаем в данных.

Длина префикса выбиралась отдельной абляцией на общей валидной подвыборке, где для всех значений `prefix_len` использовался один и тот же набор пользователей. Проверялись префиксы 25, 50, 75, 100, 150, 200 и 300 событий. На этой проверке `prefix_len = 50` давал лучший результат для retention 7d, а `prefix_len = 150` был сильнейшим вариантом для чистого RNN-эмбединга на retention 14d. Поэтому `prefix_len = 150` выбран как компромисс: он не является лучшим по всем одиночным метрикам, но дает сильное качество на более длинном горизонте и сохраняет достаточно контекста для финальной оценки устойчивости.

Финальная оценка устойчивости проводилась на `prefix_len = 150` и пяти случайных инициализациях XGBoost: 11, 42, 77, 123, 202.

Таблица 5 – Финальная оценка устойчивости RNN-эмбедингов в prefix-based режиме

Target	Feature set	ROC-AUC	PR-AUC	Test users
retention_7d	Baseline	0.6609 ± 0.0022	0.4551 ± 0.0042	2,805
retention_7d	Baseline + GRU h256 l1 max	0.6922 ± 0.0052	0.4978 ± 0.0023	2,805
retention_14d	Baseline	0.6788 ± 0.0027	0.3412 ± 0.0043	2,303
retention_14d	Baseline + GRU h256 l1 max	0.7051 ± 0.0032	0.3644 ± 0.0037	2,303

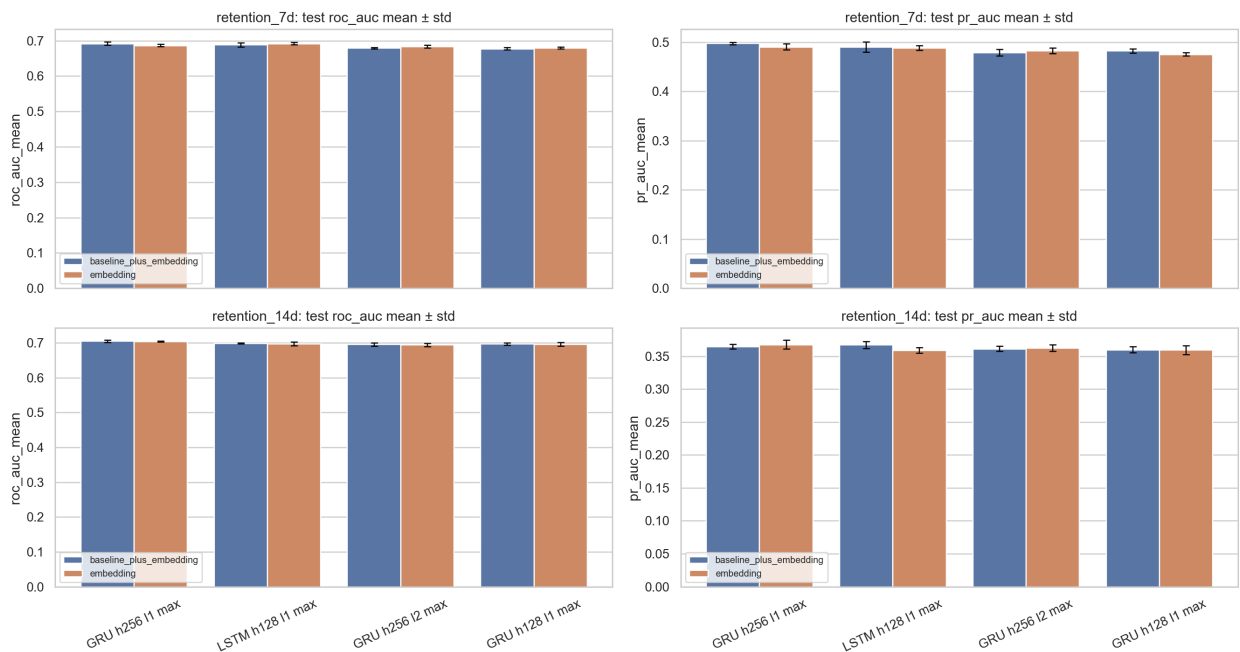


Рисунок 1 – Устойчивость финальных RNN-кандидатов в задаче удержания prefix-based.

Прирост к базовым признакам составляет примерно +0.031 ROC-AUC для retention 7d и +0.026

ROC-AUC для retention 14d. В этой постановке RNN-эмбединг не заменяет простые агрегаты поведения: длина префикса, число уникальных событий и доли частых действий уже дают заметную часть качества. Добавление эмбединга улучшает ранжирование, значит в скрытом состоянии остается информация, которой нет в этих агрегатах.

7.3 Pooling и capacity

Ablation способа pooling проверяла, как способ объединения скрытых состояний влияет на качество оценки удержания. Для GRU h128 лучшим вариантом без дополнительных baseline-признаков оказалась max pooling:

Таблица 6 – Лучшие результаты абляции pooling для GRU h128

Target	Лучший способ pooling	ROC-AUC	PR-AUC
retention 7d	GRU h128 max	0.6827	0.4800
retention 14d	GRU h128 max	0.7025	0.3663

Перебор ёмкости модели показал, что качество next-event prediction и качество оценки удержания не полностью совпадают. GRU h256 l2 дает лучший MRR в предсказании следующего события (0.8925), но downstream-таблица для удержания выбирает другие варианты. Для retention 7d лучший одиночный результат в таблице дает LSTM h128 l1 max, а для retention 14d - GRU h256 l1 max.

Таблица 7 – Основные выводы перебора ёмкости модели

Эксперимент	Лучший вариант	Метрика
next-event prediction	GRU h256 l2	MRR 0.8925
retention_7d, Baseline + эмбединг	LSTM h128 l1 max	ROC-AUC 0.6909
retention_14d, Baseline + эмбединг	GRU h256 l1 max	ROC-AUC 0.7100
final stability	GRU h256 l1 max	выбран как основной кандидат

Для дальнейших экспериментов выбран GRU h256, 1 layer, max pooling. В этой задаче важны не только последние события префикса, но и сильные активации на разных его участках; поэтому max pooling оказался полезнее вариантов, которые сильнее усредняют историю. Конфигурация выбрана как практический компромисс по двум retention-горизонтам, а не только по MRR в задаче следующего события.

7.4 Supervised fine-tuning и negative controls

С помощью supervised fine-tuning проверялось, дает ли прямое обучение под retention прирост относительно RNN-эмбеддингов, полученных из next-event prediction. Для retention 7d дообученная GRU достигла ROC-AUC 0.6882. Это близко к лучшим вариантам без разметки удержания, поэтому в финальной конфигурации основной источник представлений оставлен без supervised-дообучения.

Negative controls разделяют несколько возможных объяснений результата. Random embeddings проверяют, не получает ли XGBoost качество просто из-за добавления плотного вектора. Bag-of-events оценивает вклад состава событий без учета их порядка. Shuffled-order embeddings сохраняют тот же набор токенов, но нарушают последовательность, поэтому показывают, насколько порядок событий влияет на итоговое представление. В контрольных экспериментах:

- random embeddings дают ROC-AUC около 0.51 для retention 7d и около 0.50 для retention 14d;
- bag-of-events конкурентоспособен, но слабее основных RNN-представлений;
- shuffled-order embeddings сохраняют часть качества, что объясняется повторяемостью пользовательских префиксов, но уступают нормальным RNN-эмбеддингам.

Таблица 8 – Negative controls для оценки удержания

Feature set	Target	ROC-AUC	PR-AUC
Random embeddings	retention_7d	0.5104	0.2818
GRU mean	retention_7d	0.6756	0.4812
Baseline + GRU mean	retention_7d	0.6751	0.4757
Random embeddings	retention_14d	0.4982	0.1818
GRU mean	retention_14d	0.6937	0.3604
Baseline + GRU mean	retention_14d	0.6979	0.3663

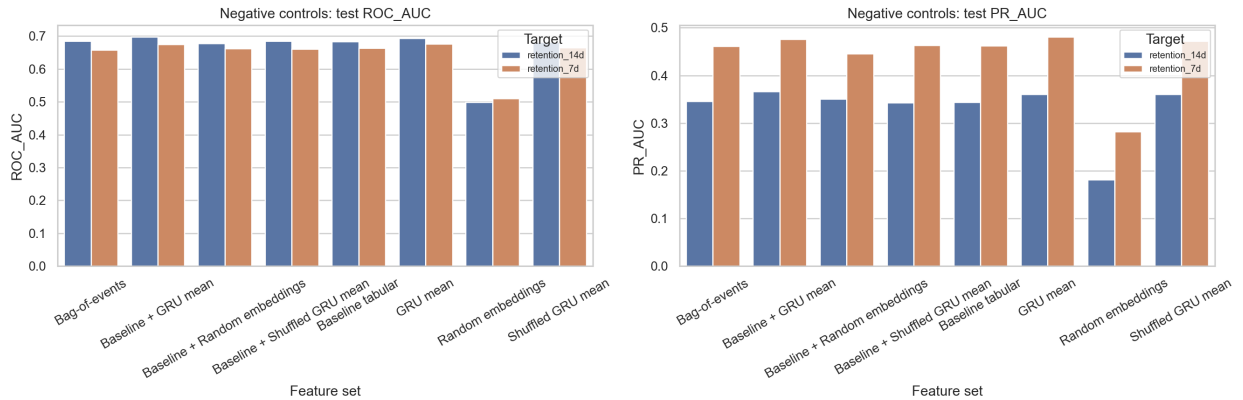


Рисунок 2 – Сравнение RNN-эмбеддингов с отрицательными контролями на тестовой части.

Сравнение с контролями показывает, где именно появляется выигрыш RNN. Случайный плотный вектор почти не отличается от случайного ранжирования, значит сама размерность признакового блока не объясняет результат. Bag-of-events и shuffled-order ближе к RNN, потому что сохраняют состав пользовательских действий. Разрыв между ними и обычной RNN означает, что часть качества связана с последовательной структурой префикса, а не только с частотами событий.

7.5 Full-history сравнение

После экспериментов prefix-based была добавлена RNN-ветка full-history. Она использует тот же командный split, но строит эмбединг по последним 512 событиям полной истории пользователя. Для командного сравнения эта ветка была приведена к общему протоколу: RNN обучается на тех же SimCLR processed sequences, использует те же train/val/test user ids и те же duration labels из SimCLR-пакета.

Этот режим нужен для командного сопоставления с SimCLR/BERT4Rec, где другие ветки также работали с представлениями full-history. Его нельзя напрямую сравнивать с результатами prefix-based как с той же задачей раннего прогноза.

Таблица 9 – Оценка RNN-эмбедингов full-history

Target	Feature set	ROC-AUC	PR-AUC	Test users
retention_1d	RNN embedding	0.9598 ± 0.0003	0.8818 ± 0.0012	10,260
retention_1d	Baseline + RNN embedding	0.9641 ± 0.0003	0.9005 ± 0.0010	10,260
retention_7d	RNN embedding	0.9413 ± 0.0005	0.6777 ± 0.0027	10,260
retention_7d	Baseline + RNN embedding	0.9471 ± 0.0006	0.7170 ± 0.0032	10,260
retention_14d	RNN embedding	0.9319 ± 0.0012	0.4960 ± 0.0089	10,260
retention_14d	Baseline + RNN embedding	0.9388 ± 0.0009	0.5565 ± 0.0076	10,260
retention_30d	RNN embedding	0.9658 ± 0.0027	0.1910 ± 0.0456	10,260
retention_30d	Baseline + RNN embedding	0.9757 ± 0.0014	0.2381 ± 0.0465	10,260

Метрики full-history значительно выше, потому что модель видит всю доступную историю пользователя. Это показывает, что RNN хорошо кодирует полный поведенческий профиль, но не заменяет prefix-based проверку, если интересуется ранний прогноз будущего поведения. В согласованном прогоне используется та же тестовая часть на 10 260 пользователей, что и в командных экспериментах full-history.

7.6 Командное сравнение подходов full-history

В групповой части проекта, помимо RNN-ветки, были реализованы еще два подхода к построению пользовательских эмбедингов. SimCLR-ветка обучала контрастивный энкодер: две аугменти-

рованные версии истории одного пользователя сближались в пространстве эмбедингов, а истории разных пользователей отталкивались друг от друга. Transformer/BERT4Rec-ветка обучала последовательностный энкодер в постановке маскирования и предсказания событий, а затем экспортировала эмбединги по последним 512 событиям полной истории.

Для сопоставления подходов используется сравнение эмбедингов без дополнительных baseline-признаков в общей постановке full-history. Метрики SimCLR взяты из экспортированных артефактов соответствующей ветки, метрики BERT - из итоговой сводки BERT-ветки, а метрики RNN - из согласованного прогона full-history. Во всех строках таблицы приведена оценка на 10 260 test-пользователях с одинаковыми долями положительного класса. Комбинированные варианты с ручными baseline-признаками не включаются в командную таблицу, поскольку они сильнее зависят от инженерии табличных признаков и хуже отражают качество самих эмбедингов.

Таблица 10 – Командное сравнение эмбедингов full-history без дополнительных baseline-признаков

Горизонт	Модель	ROC-AUC	PR-AUC	Пользователей	Доля positive
7d	BERT mean	0.9296	0.6228	10,260	11.1%
7d	RNN/GRU	0.9413	0.6777	10,260	11.1%
7d	SimCLR	0.9249	0.6263	10,260	11.1%
14d	BERT mean	0.9256	0.4467	10,260	6.0%
14d	RNN/GRU	0.9319	0.4960	10,260	6.0%
14d	SimCLR	0.9189	0.4974	10,260	6.0%
30d	BERT mean	0.9665	0.1500	10,260	0.34%
30d	RNN/GRU	0.9658	0.1910	10,260	0.34%
30d	SimCLR	0.8621	0.1725	10,260	0.34%

В текущей таблице, выровненной по full-history постановке, test-части и retention-меткам, RNN/GRU дает лучший ROC-AUC среди результатов без дополнительных baseline-признаков на горизонтах 7 и 14 дней. На 30 днях BERT mean и RNN/GRU практически равны по ROC-AUC, а PR-AUC выше у RNN. При этом 30d нужно читать осторожнее: положительный класс составляет всего около 0.34% тестовой части.

Совпадение тестовой части, определения retention-меток и долей положительного класса делает таблицу содержательным сравнением для курсового проекта. При этом ветки реализовывались независимо, поэтому таблица не утверждает полного равенства кода оценки во всех подходах.

7.7 Анализ пространства эмбедингов

Для анализа пространства эмбедингов были построены нормы, PCA-проекции и KMeans-профили. GRU и LSTM формируют невырожденные пространства представлений:

Таблица 11 – Диагностика пространства GRU и LSTM эмбедингов

Модель	Mean norm	Median norm	PCA10 cumulative variance
GRU	8.4440	8.4336	0.6053
LSTM	6.9537	7.0392	0.5779

Первые 10 PCA-компонент объясняют около 60% дисперсии для GRU и около 58% для LSTM. Это говорит о выраженной структуре пространства. Кластеризация по GRU-эмбедингам выделяет группы пользователей с различающимися profile statistics и retention rates.

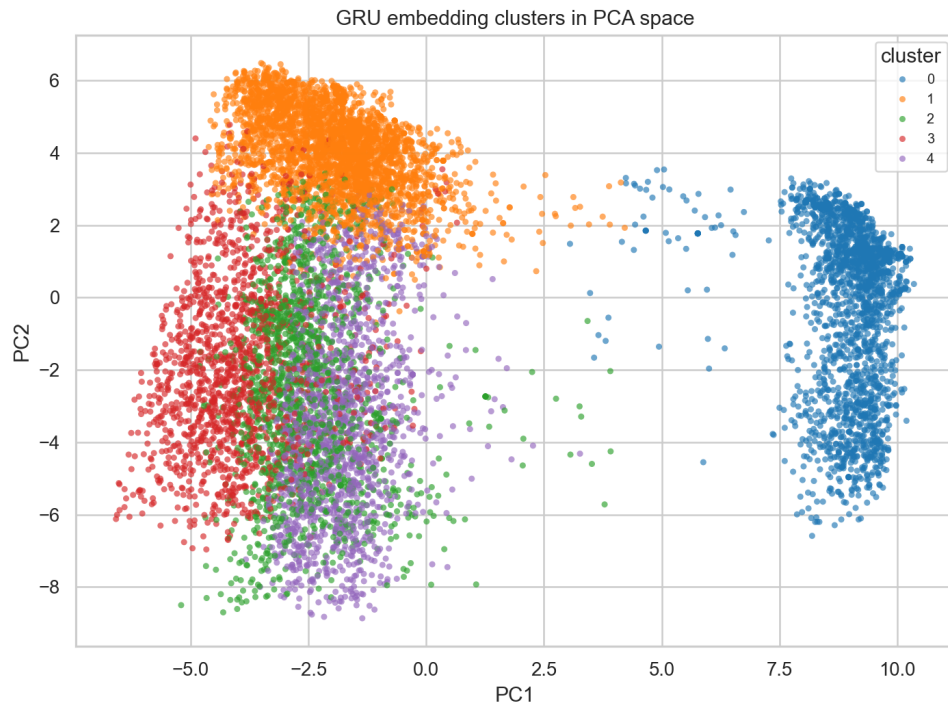


Рисунок 3 – PCA-проекция GRU-эмбедингов с KMeans-кластерами пользователей.

8 Воспроизводимость и проверка корректности

Для снижения риска утечек и смешивания постановок в работе отдельно реализованы сценарии prefix-based и full-history. В prefix-based режиме все признаки, включая baseline-агрегаты и RNN-эмбединги, строятся только по доступному префиксу пользователя. В full-history режиме используется отдельная разметка и отдельный экспорт эмбедингов по последнему окну истории.

Выбор top-K событий для baseline-признаков выполняется только на train-части. Модели оценки обучаются на train и оцениваются на val/test, что сохраняет разделение данных по пользователям.

Для финальных таблиц используются несколько случайных инициализаций XGBoost, поэтому в отчете приводятся не только средние значения метрик, но и стандартные отклонения.

Экспериментальные артефакты проверяются отдельным gunner: он валидирует наличие ключевых таблиц, графиков, manifest-файлов и согласованных результатов full-history, а также формирует сводную таблицу результатов. В текущей версии gunner проверяет 57 артефактов по 10 сериям экспериментов. Ноутбуки выполнены без сохраненных error outputs, а исходный код проходит синтаксическую проверку.

Таким образом, основная часть контроля качества относится не к структуре репозитория, а к корректности экспериментального протокола: отдельные режимы оценки, user-level split, отсутствие использования test-части при построении baseline-признаков и воспроизводимая проверка итоговых артефактов.

9 Заключение

В этой части проекта построены пользовательские эмбединги по последовательностям игровых событий с помощью GRU и LSTM. Энкодеры обучались на next-event prediction без retention-разметки; метки удержания использовались только в downstream-оценке.

В задаче предсказания следующего события рекуррентные модели обошли MostPopular и Markov-1. Для GRU на тестовой части получены MRR 0.8885 и Hit@10 0.9926. Это означает, что скрытое состояние RNN кодирует переходы между событиями, а не только частоты токенов.

Основная проверка без утечки проводилась в prefix-based режиме: эмбединг строился по началу истории, а удержание считалось относительно конца префикса. Финальный кандидат GRU h256, 1 layer, max pooling получил ROC-AUC 0.6922 ± 0.0052 для retention 7d и 0.7051 ± 0.0032 для retention 14d. Это оценка раннего представления пользователя, поскольку будущие события не попадают в признаки.

Отдельно выполнен full-history прогон для командного сравнения с SimCLR и BERT4Rec. В нем RNN строит эмбединг по последним 512 событиям полной истории и оценивается на 10 260 test-пользователях. В этой постановке получены ROC-AUC 0.9413 для retention 7d и 0.9319 для retention 14d. Этот результат нельзя напрямую переносить на prefix-based режим, потому что full-history использует уже наблюдавшийся полный профиль пользователя.

Ветки SimCLR, BERT4Rec и RNN разрабатывались независимо, поэтому full-history таблицу следует читать как командное сравнение в согласованной постановке, а не как полностью контролируемую абляцию. Для финального сопоставления были выровнены split, test-часть и определение retention-меток.

Список использованных источников

- [1] Bengio Y., Courville A., Vincent P. Representation Learning: A Review and New Perspectives // IEEE Transactions on Pattern Analysis and Machine Intelligence. 2013. Vol. 35, No. 8. P. 1798-1828. DOI: 10.1109/TPAMI.2013.50. URL: <https://doi.org/10.1109/TPAMI.2013.50>.
- [2] Chen T., Kornblith S., Norouzi M., Hinton G. A Simple Framework for Contrastive Learning of Visual Representations // Proceedings of the 37th International Conference on Machine Learning. PMLR. 2020. Vol. 119. P. 1597-1607. URL: <https://proceedings.mlr.press/v119/chen20j.html>.
- [3] Chen T., Guestrin C. XGBoost: A Scalable Tree Boosting System // Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2016. P. 785-794. DOI: 10.1145/2939672.2939785. URL: <https://doi.org/10.1145/2939672.2939785>.
- [4] Cho K., van Merriënboer B., Gulcehre C., Bahdanau D., Bougares F., Schwenk H., Bengio Y. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation // Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing. 2014. P. 1724-1734. DOI: 10.3115/v1/D14-1179. URL: <https://aclanthology.org/D14-1179/>.
- [5] Hidasi B., Karatzoglou A., Baltrunas L., Tikk D. Session-based Recommendations with Recurrent Neural Networks // ICLR. 2016. arXiv:1511.06939. URL: <https://arxiv.org/abs/1511.06939>.
- [6] Hochreiter S., Schmidhuber J. Long Short-Term Memory // Neural Computation. 1997. Vol. 9, No. 8. P. 1735-1780. DOI: 10.1162/neco.1997.9.8.1735. URL: <https://doi.org/10.1162/neco.1997.9.8.1735>.
- [7] van den Oord A., Li Y., Vinyals O. Representation Learning with Contrastive Predictive Coding. 2018. arXiv:1807.03748. URL: <https://arxiv.org/abs/1807.03748>.
- [8] Sun F., Liu J., Wu J., Pei C., Lin X., Ou W., Jiang P. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer // Proceedings of the 28th ACM International Conference on Information and Knowledge Management. 2019. P. 1441-1450. DOI: 10.1145/3357384.3357895. URL: <https://doi.org/10.1145/3357384.3357895>.